Print a Value

You'll naturally want to use `p` and the related methods to print values from specific points in the app. Some techniques that are useful here:

- Print the arguments to a method call.
- Print an ActiveRecord object before and after an update.
- Print both sides of a compound Boolean separately.

And so on. If you're particularly industrious, use Ruby string interpolation to give a label—`p "user name: #{user.name}"`—but for most quick uses you don't need to bother.

Deface a Gem

You're not limited to your own application when inserting print statements. It's easy and fun to insert code into a gem (just remember to undo it).

Bundler has two commands: `bundle show` and `bundle open`. The `bundle show` command returns the complete path to the version of the gem that Bundler is using. If you want to go one step further, `bundle open` will open the gem directory in your default editor.

Add print or log statements to taste. This can be useful when first interacting with a new gem. You'll need to restart your server or Spring instance for the changes to take effect.

When you're done, use the command `gem pristine <gemname>` to return the gem to its original state. Or reinstall the bundle.

Show a Page

It's very important to see what's going on inside an integration test that uses Capybara or Cucumber. Capybara has the helper method `save_and_open_page`, which does exactly what it says. It dumps the current Capybara DOM to a temp file and opens the file in your default browser.

The resulting browser window will look a little strange. Any relative files or assets won't be displayed, meaning no CSS and only some images. But the text is usually enough to see if your integration test is even looking at the page you expect, and not, say, looking at the login page because you forgot to log in. The gem capybara-screenshot will do this automatically on any failing Capybara test.[4]

4. https://github.com/mattheworiordan/capybara-screenshot